

Naslov: Diferencijalna evolucija



1. Uvod

Diferencijalna evolucija (DE) optimizaciona metaheuristika, podvrsta evolucionih algoritama, i pripada grupi populacionih metaheuristika, koja održava populaciju potencijalnih rešenja za optimizacioni problem koji se rešava i iterativno pokušava da ih unapredi u odnosu na unapred zadatu meru kvaliteta. Diferencijalna evolucija nema gotovo nikakve pretpostavke o problemu koji rešava, ali takođe nema nikakve garancije da će optimalno rešenje ikada biti pronađeno.

Najčešći slučajevi upotrebe za diferencijalnu evoluciju su višedimenzionalni problemi kontinualne optimizacije. Kako diferencijalna evolucija ne koristi gradijent, optimizacioni problemi koje rešava ne moraju biti diferencijabilni, a čak ne moraju biti ni neprekidni. Zbog toga, diferencijalna evolucija pogoduje za rešavanje najtežih optimizacionih problema, uključujući i probleme sa šumom, ili probleme koji se menjaju kroz vreme. [1][2]

Prvi put diferencijalna evolucija se pojavljuje 1995. u radu Rajnera Storna i Keneta Prajsa. Od tada su napisane brojne knjige i radovi na temu diferencijalne evolucije koji se bave različitim teoretskim i praktičnim aspektima diferencijalne evolucije. [4][5]

Iako je diferencijalna evolucija doživela pravu evoluciju, sa raznim novim i hibridnim varijantama algoritma, u osnovi diferencijalne evolucije je ostao jednostavan algoritam. Za svakog člana populacije generiše se novi kandidat, nakon čega se vrši poređenje. Ključna karakteristika diferencijalne evolucije je generisanje kandidata, koje se sastoji iz izbora slučajnih članova populacije, njihovog spajanja u prototip kandidata, i slučajnog izbora karakteristika starog člana koji će novi zadržati. [9]

Pored kontinualnih, diferencijalna evolucija se može koristiti i za probleme diskretne optimizacije. Interno, algoritam i dalje koristi neprekidne vrednosti za kandidate rešenja, a adaptacija se događa u ciljnoj funkciji, odnosno funkciji cene koja zaokružuje ulazne vrednosti do najbližeg celog broja i onda sa zaokruženim vrednostima računa kvalitet rešenja. [3]

2. Algoritam

Diferencijalna evolucija pokušava da unapredi svaku jedinku u populaciji tako što će iskombinovati nekoliko drugih jedinki iz populaciju u skladu sa nekim zadatim jednostavnim pravilom. U svojoj osnovnoj verziji algoritam ima dva parametra, i to su verovatnoća krossovera p i diferencijalna težina ω . Pored ovih parametara, imamo i n_{pop} parametar za veličinu populacije, koja se nasumično

inicijalizuje pre početka rada algoritma. Algoritam se sastoji iz ponavljanja sledećih koraka za svaku jedinku u populaciji. [4][5]

1. Nasumično izabrati tri različite jedinke $a, b \text{ i } c$
2. Konstruisati privremenu jedinku $z = a + \omega \cdot (b - c)$
3. Slučajno izabrati dimenziju $j \in [1, \dots, n_{param}]$,
gde je n_{param} broj dimenzija jedinke, i slučajan broj $r \in [0, 1]$
4. Konstruisati jedinku kandidata x' koristeći binarni crossover
$$x'_i = \begin{cases} z_i, & \text{ako } i = j \text{ ili } r < p \\ x_i, & \text{inače} \end{cases}$$

gde $i \in [1, \dots, n_{param}]$
5. Izabrati bolju jedinku od $x \text{ i } x'$ za sledeću generaciju

Nakon formiranja nove generacije populacije, prelazi se u sledeću iteraciju, do ispunjenja nekog uslova zaustavljanja, koji takođe mogu imati svoje parametre. [4][5]

Primetimo da se istraživanje dešava od prvog do četvrtog koraka, a da je peti korak pohlepan i tu se dešava eksploatacija. Zbog pohlepnog petog koraka, algoritam nikada neće prihvatiti lošije rešenje, tako da će se populacija uvek monotono kretati ka optimumu. Sa druge strane, probna jedinka se ne poredi sa celom tekućom populacijom, već samo sa njenim parom iz tekuće populacije, čime se smanjuje pohlepnost petog koraka.

3. Izbor p i ω kontrolnih parametara

Obično veće vrednosti n_{pop} i ω rezultuju robusnom pretragom, ali po ceni sporije konvergencije. Premale vrednosti n_{pop} i ω će dovesti do preuranjene konvergencije ili stagnacije pre pronalaženja globalnog optimuma. [3]

Uobičajeno n_{pop} varira između $2n_{param}$ i $20n_{param}$, mada neki izrazito multimodalni problemi mogu zahtevati i više od $50n_{param}$, dok se za unimodalne i lakše multimodalne probleme preporučuju manje populacije. [3]

Iskustvo govori da vrednosti ω između $2/n_{pop}$ i $1,2$ daju najbolje rezultate. Konkretnu vrednost je potrebno uskladiti sa osobinama ciljne funkcije koja se optimizuje. [3]

Ako nije poznato da je ciljna funkcija razdvojiva, nizak prag verovatnoće p između 0 i $0,1$ je dobar izbor početne vrednosti. Za prag verovatnoće $p = 0$ algoritam je rotaciono invarijantan. [3]

4. Druge varijante diferencijalne evolucije

Vremenom, pojavile su se brojne varijante i modifikacije osnovnog algoritma diferencijalne evolucije. Veliki deo tih modifikacija se odnosio na broj i način izbora jedinki, kao i način konstrukcije jedinke u prva dva koraka. U cilju klasifikacije različitih varijanti, uvedena je notacija

DE/x/y/z,

gde x predstavlja način izbora jedinke koja će biti mutirana, y je broj razlika jedinki koje učestvuju u mutaciji, a z predstavlja crossover shemu. U skladu sa ovom notacijom, osnovna strategija diferencijalne evolucije je označena sa DE/rand/1/bin. [6]

Jedna od popularnih varijacija je DE/best/2/bin, gde mutiramo najbolju jedinku u tekućoj populaciji sa dve razlike jedinki.

$$z = best + \lambda \cdot (a - b) + \omega \cdot (c - d)$$

Tokom prvog koraka ne biramo tri, već četiri jedinke, i uvodimo još jedan parametar λ . Za dovoljno velike populacije, upotreba dve razlike jedinki može da poboljša diverzitet populacije. [6] Mutiranje isključivo najbolje jedinke čini algoritam pohlepni i doprinosi eksploataciji, ali krossover sa tekućom jedinkom ublažava pohlepnost.

5. Samoadaptivni algoritam

Velika prednost algoritma diferencijalne evolucije je što ima mali broj parametara. Podešavanje parametara samo po sebi podrazumeva da imamo neka znanja o ciljnoj funkciji, ili da moramo da se stavimo u ulogu optimizatora i pretražujemo prostor parametara. Tada se pojavila ideja da pustimo optimizator da sam pretražuje prostor parametara.

Stohastička diferencijalna evolucija (SDE) predstavlja samoadaptivnu modifikaciju algoritma diferencijalne evolucije koja sama pretražuje prostor parametara za prag verovatnoće p i diferencijalnu težinu ω . Osnovna varijanta stohastičke diferencijalne evolucije predstavlja modifikaciju strategije DE/rand/1/bin, koja za svaku jedinku u populaciji generiše parametre p i ω . [7]

Na početku algoritma inicijalizujemo posebnu populaciju Ω , tako što za svaku jedinku u populaciji inicijalizujemo ω slučajnom vrednošću iz normalne raspodele $N(0,5, 0,15)$, koje bi uglavnom trebalo da se nalaze u intervalu $[0, 1]$. Zatim, pre drugog koraka generišemo faktor ω za svaku jedinku u populaciji.

$$\omega = \omega_1 + \mathcal{N}(0, 0.5) \cdot (\omega_2 - \omega_3)$$

gde su parametri ω_1, ω_2 i ω_3 slučajno izabrani iz Ω . [7]

Zatim, u četvrtom koraku, umesto parametra praga verovatnoće p uzima se vrednost iz normalne distribucije $N(0,5, 0,15)$, tako da umesto uslova $r < p$, sada koristimo uslov $r < N(0,5, 0,15)$, gde r dolazi iz uniformne raspodele $U(0, 1)$. [7]

Na taj način smo se oslobodili kontrolnih parametara p i ω . Iako je stohastička diferencijalna evolucija prvobitno definisana kao proširenje DE/rand/1/bin strategije, nema razloga zašto ovaj pristup ne bi mogao da se primeni i za druge strategije. Na primer, za DE/best/2/bin bi na sličan način generisali i λ kontrolni parameter. Dalje, nove modifikacije bi mogle da predlože nove načine generisanja težinskih faktora.

Naprednije samoadaptivne varijante samostalno podešavaju i n_{pop} veličinu populacije, u zavisnosti od nekih statistika populacije, uz sofisticiranije metode automatskog podešavanja kontrolnih parametara. Takve varijante su znatno kompleksnije, uvode dosta novih modifikacija i mogu se razmatrati kao poseban algoritam, premda u osnovni zadržavaju bazne korake diferencijalne evolucije. [8][9]

6. Ograničenja domena pretrage

Kada govorimo o ograničenjima domena pretrage, razmatraju se dve vrste ograničenja, meka i čvrsta ograničenja.

Čvrsta ograničenja predstavljaju okvire prostora pretrage iz kojih jedinke ne smeju da istupe. Ukoliko se prilikom generisanja jedinke otkrije prestup čvrstih ograničenja, modifikuju se tako da se vrate u zadate okvire.

Čvrsta ograničenja se obično zadaju na nivou parametara jedinke, tako da se vrednost parametara koja izađe iz okvira dozvoljenih vrednosti menja nasumično izabranom vrednošću iz okvira parametara. [3]

$$x'_i = \begin{cases} x_{i,lower} + rand \cdot (x_{i,upper} - x_{i,lower}), & \text{ako } x'_i < x_{i,lower} \text{ ili } x_{i,upper} < x'_i \\ x'_i, & \text{inače} \end{cases}, i \in [1, n_{param}]$$

Meka ograničenja, sa druge strane, su granice koje dopuštamo jedinkama da pređu, ali penalizujemo njihov prelazak. Obično se zadaju funkcijama koje vraćaju neutral za dozvoljene vrednosti jedinke, a za nedozvoljene vrednost penala.

Funkcija cene objedinjuje funkcije ograničenja sa ciljnom funkcijom. Za definisanje funkcije cene obično se koriste aditivni i multiplikativni pristup. Aditivni pristup podrazumeva da se vrednosti koje vrate g funkcije ograničenja sabere sa vrednošću koju vrati $f_{objective}$ ciljna funkcija.

$$f_{cost}(x) = f_{objective}(x) + \sum_{i=1}^{n_{param}} g_i(x)$$

Multiplikativni pristup obično podrazumeva množenje vrednosti ciljne funkcije sa stepenovanim vrednostima funkcija ograničenja. Neutral za aditivni pristup nula, a neutral za multiplikativni je jedan. Pored toga što u multiplikativnom pristupu moramo da obezbedimo da funkcije ograničenja vraćaju jedinicu ako je sve uredu, moramo da obezbedimo i da ciljna funkcija konvergira ka jedinici sa desne strane. Time unosimo dodatne pretpostavke o funkciji, tako da se multiplikativni pristup retko koristi.

7. Uslovi zaustavljanja

Sam algoritam diferencijalne evolucije ne definiše uslove zaustavljanja, već prepušta korisniku da izabere uslov zaustavljanja u skladu sa potrebama problema koji rešava.

Često korišćeni uslovi zaustavljanja su dostizanje maksimalnog broja iteracija, dostizanje neke vrednosti ciljne funkcije, broj iteracija bez napretka, ili vremensko ograničenje. Ove uslove je moguće i kombinovati, čime nastaju kompleksniji uslovi zaustavljanja koji prevazilaze nedostatke pojedinačnih uslova zaustavljanja. Postoje i uslovi zasnovani na statističkim testovima, koji proveravaju kakve su šanse da u daljim iteracijama dođe do unapređenja. [2]

Kako sam algoritam diferencijalne evolucije ne definiše eksplicitno uslove zaustavljanja, u ovom radu se nećemo detaljnije baviti uslovima zaustavljanja.

8. Implementacija

Za implementaciju korišćen je modularni, parametrizovani pristup, gde imamo jednu klasu u kojoj je implementiran generički algoritam diferencijalne evolucije, a konkretne funkcije za selekciju, mutaciju i krossover se prosleđuju kao parametri prilikom pokretanja. Time je obezbeđena proširiva implementacija, pogodna za pokretanja testova.

Osnovu implementacije predstavlja klasa *DifferentialEvolution* koja se inicijalizuje sa ciljnom funkcijom, funkcijama ograničenja, čvrstim granicama i veličinom populacije.

```
class DifferentialEvolution:
    def __init__(self,
                  objective_function,
                  constraint_functions,
                  upper_bounds,
                  lower_bounds,
                  population_size):
```

Prilikom pokretanja diferencijalne evolucije definišemo koji uslov zaustavljanja želimo da koristimo, kao i funkcije za krossover, selekciju i mutaciju. Na taj način za konkretan problem možemo da brzo i lako da isprobamo više različitih strategija i odaberemo najpovoljniji rezultat.

```
    def run(self,
            stopping_condition,
            crossover,
            selection_mutation):
```

Operacije selekcije jedinki i same mutacije su objedinjene u jednu funkciju, jer su te operacije međusobno uslovljene. Izbor strategije mutacije određuje koje vrednosti želimo da dobijemo selekcije. Ukoliko koristimo strategiju DE/rand/1/bin, onda u mutaciji koristimo tri slučajno izabrane jedinke. Za strategiju DE/best/2/bin želimo da nam selekcija da najbolju jedinku i četiri slučajno izabrane jedinke. Strategija DE/best/1/bin bi zahtevala da selekcija vrati najbolju i dve slučajne jedinke. Stoga, ima smisla ove operacije objediniti u jednoj funkciji.

Drugo odstupanje od kanonske diferencijalne evolucije je računanje krossover vektora na početku iteracije. To nam omogućuje da prilikom mutacije uštedimo na računanju parametara za koje znamo da će biti odbačeni.

```
    def initialize_stats(self):
        self.iter_count = 0
        self.eval_count = 0
        self.best_solution_hist = []
        self.best_score_hist = []
        self.population_diversity_hist = []
```

Tokom izvršavanja algoritma održavaju se statistike poput broja iteracija, poziva funkcije cene, kao i istorija najboljih rešenja, ocena i raznolikosti populacije. Raznolikost se računa kao prosek sume euklidskog rastojanja za svake jedinku u populaciji.

```
    def diversity(self, population):
        n = self.population_size
        total_dist = 0

        for a, b in combinations(population, r=2):
            total_dist += sqrt(sum([(x - y)**2 for x, y in zip(a, b)]))

        return total_dist/(n*(n-1)/2)
```

Nakon implementacije osnovnih uslova zaustavljanja, implementirane su funkcije za strategije DE/rand/1/bin i DE/best/2/bin, kao i funkcije za SDA modifikaciju.

```
class CrossoverSA:
    def do(self, x):
        d = random.randrange(len(x))
        u = [random.random() < random.gauss(0.5, 0.15) for _ in x]
        u[d] = True

        return u
```

Specifičnost ove samoadaptivne implementacije je da se kontrolni parametri ne generišu na nivou jedinke, već na nivou parametra, tako da za svaku jedinku imamo vektor kontrolnih parametara, što doprinosi većoj raznolikosti populacije i boljem istraživanju optimizacionog prostora.

```

class SelectMutateSDE:
    def __init__(self,
                  dimension):
        self.omegas = [random.gauss(0.5, 0.15) for _ in range(dimension)]

    def do(self,
          xx,
          population,
          uu):
        self.omegas = [self.__evolve_omega() for _ in self.omegas]
        aa, bb, cc = random.sample(population, 3)

        return [a + o*(b - c) if u else x for a, b, c, o, u, x in zip(aa, bb, cc, self.omegas, uu, xx)]

    def __evolve_omega(self):
        o1, o2, o3 = random.sample(self.omegas, 3)

        return o1 + random.gauss(0, 0.5)*(o2 - o3)

```

Pored implementacije modifikovane osnovne stohastičke diferencijalne evolucije, implementirana je i samoadaptivna modifikacija strategije DE/best/2/bin.

```

class SelectMutateSDE2:
    def __init__(self,
                  dimension):
        self.nus = [random.gauss(0.5, 0.15) for _ in range(dimension)]
        self.omegas = [random.gauss(0.5, 0.15) for _ in range(dimension)]

    def do(self,
          xx,
          population,
          best,
          uu):
        self.nus = [self.__evolve_nu() for _ in self.nus]
        self.omegas = [self.__evolve_omega() for _ in self.omegas]
        aa, bb, cc, dd = random.sample(population, 4)

        return [bst + nu*(a - b) + om*(c - d) if u else x for a, b, c, d, u, x, bst, nu, om in zip(aa, bb, cc, dd, self.nus, self.omegas, uu)]

    def __evolve_nu(self):
        n1, n2, n3 = random.sample(self.nus, 3)

        return n1 + random.gauss(0, 0.5)*(n2 - n3)

    def __evolve_omega(self):
        o1, o2, o3 = random.sample(self.omegas, 3)

        return o1 + random.gauss(0, 0.5)*(o2 - o3)

```

Zatim su implementirane test funkcije i izvršeni testovi. Takođe, izvršeni su testovi i sa nekim bibliotečkim funkcijama radi poređenja rezultata.

9. Test funkcije

- Funkcija Sphere.

$$f(X) = \sum_{i=1}^n x_i^2, -\infty \leq x_i \leq \infty$$

- Funkcija Rosenbrock's.

$$f(X) = \sum_{i=1}^{n-1} 100 \cdot (x_i^2 - x_{i+1})^2 + (1 - x_i)^2, -\infty \leq x_i \leq \infty$$

- Funkcija Step.

$$f(X) = \sum_{i=1}^n [x_i], -\infty \leq x_i \leq \infty$$

- Funkcija Griewank's.

$$f(X) = 1 + \frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos\left(\frac{x_i}{\sqrt{i}}\right), -\infty \leq x_i \leq \infty$$

- Funkcija Styblinski-Tang's.

$$f(X) = \frac{\sum_{i=1}^n x_i^4 - 16x_i^2 + 5x_i}{2}, -5 \leq x_i \leq 5$$

- Funkcija Shekel's.

$$f(X) = \sum_{i=1}^m \left(c_i + \sum_{j=1}^n (x_j - a_{ji})^2 \right)^{-1}, -\infty \leq x_i \leq \infty$$

- Funkcija Rastrigin's.

$$f(X) = \sum_{i=1}^n [-10 \cos(2\pi x_i) + 10], -5.12 \leq x_i \leq 5.12$$

- Funkcija Ackley's.

$$f(X) = -20 \exp\left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}\right) - \exp\left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)\right) + 20 + \exp, -32 \leq x_i \leq 32$$

- Funkcija Rotated Ellipsoid.

$$f(X) = \sum_{i=1}^n \left(\sum_{j=1}^i x_j \right)^2, -\infty \leq x_i \leq \infty$$

- Funkcija Keane's Bump.

$$f(X) = - \left| \frac{\sum_{i=1}^m \cos^4(x_i) - 2 \prod_{i=1}^m \cos^2(x_i)}{(\sum_{i=1}^m i x_i^2)^{0.5}} \right|, 0 \leq x_i \leq 10$$

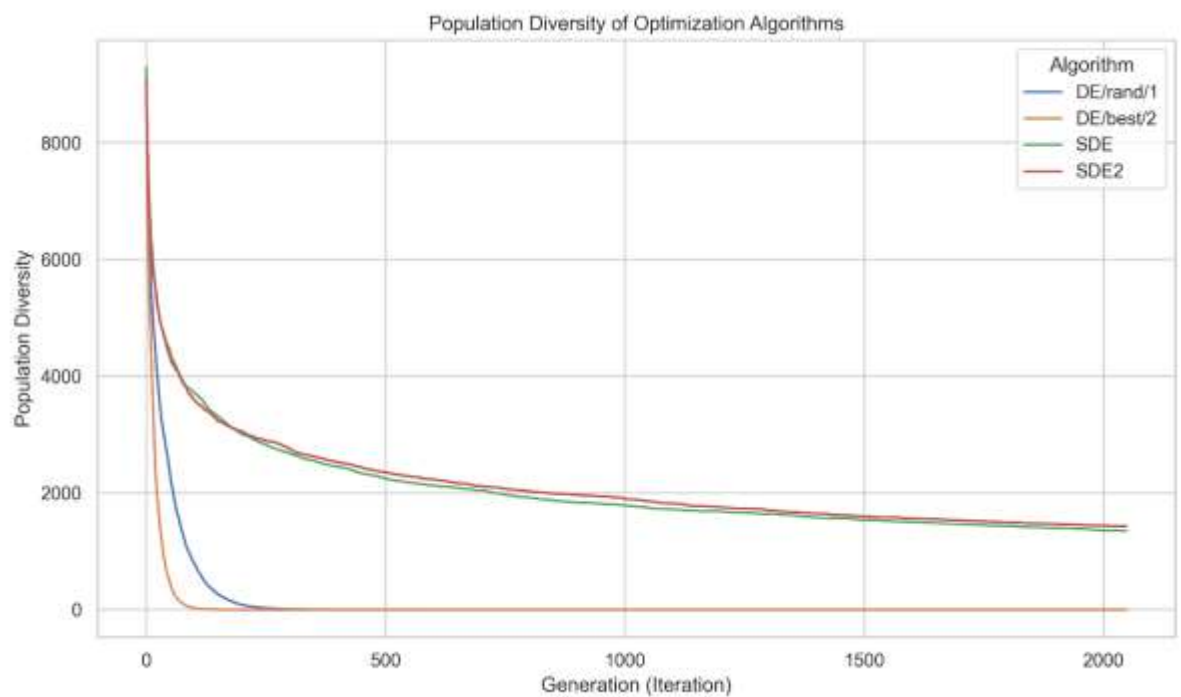
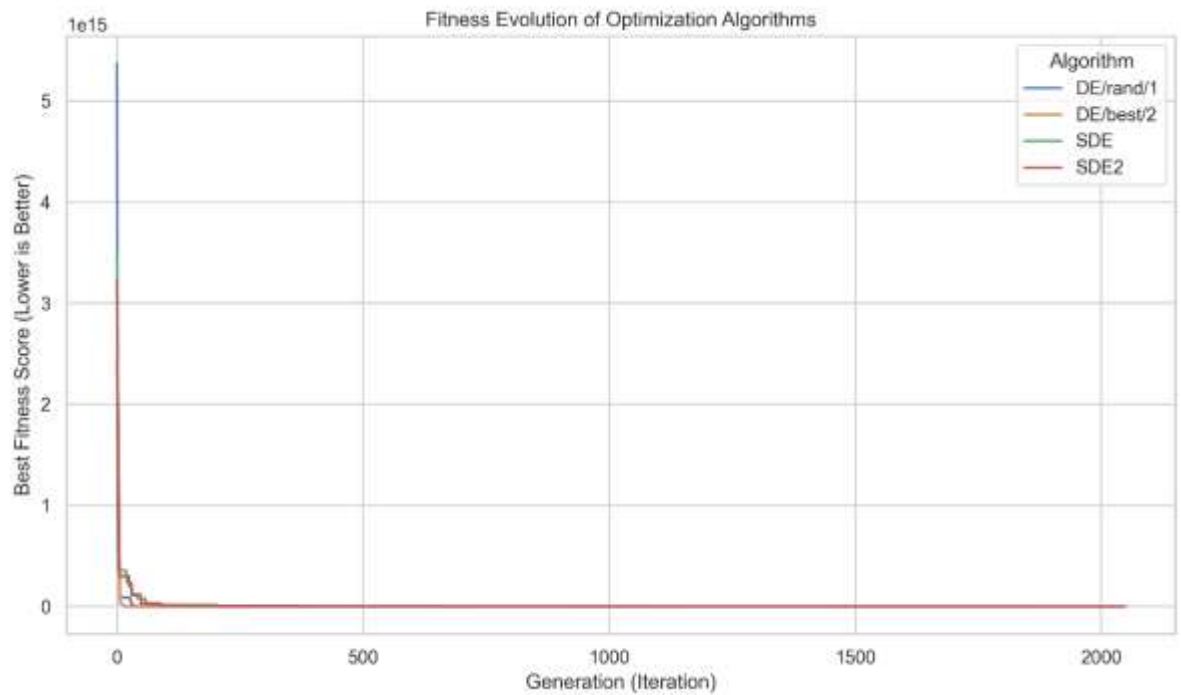
$$0.75 - \prod_{i=1}^m x_i < 0, \sum_{i=1}^m x_i - 7.5m < 0$$

10. Rezultati

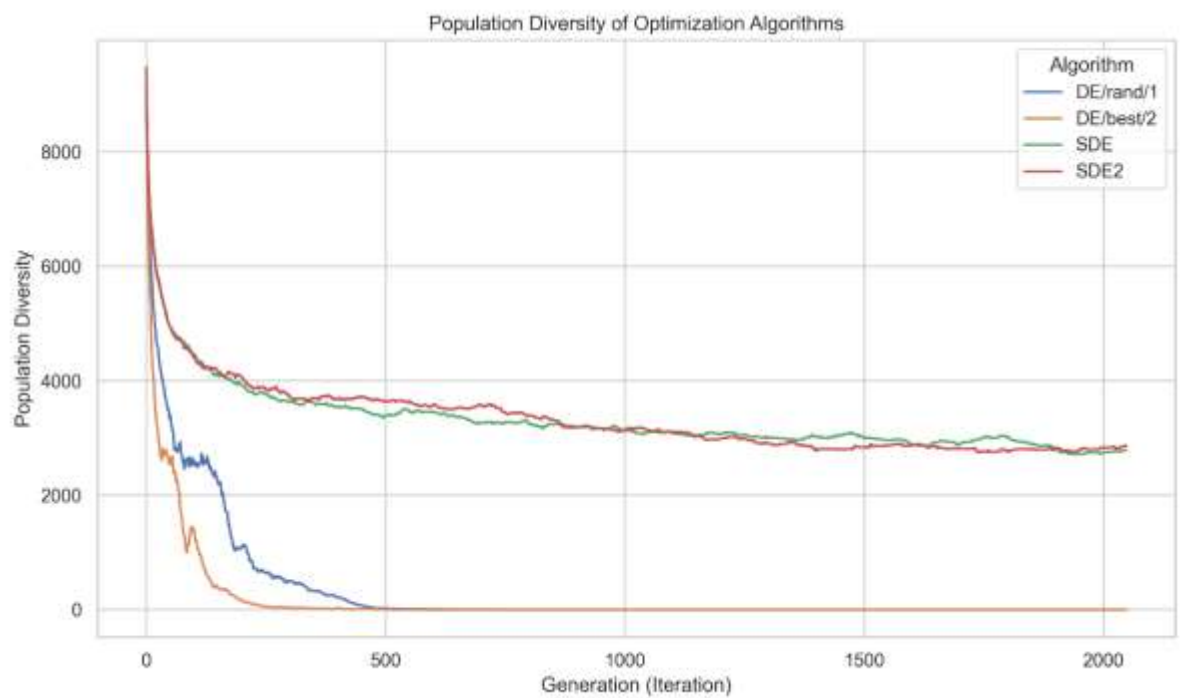
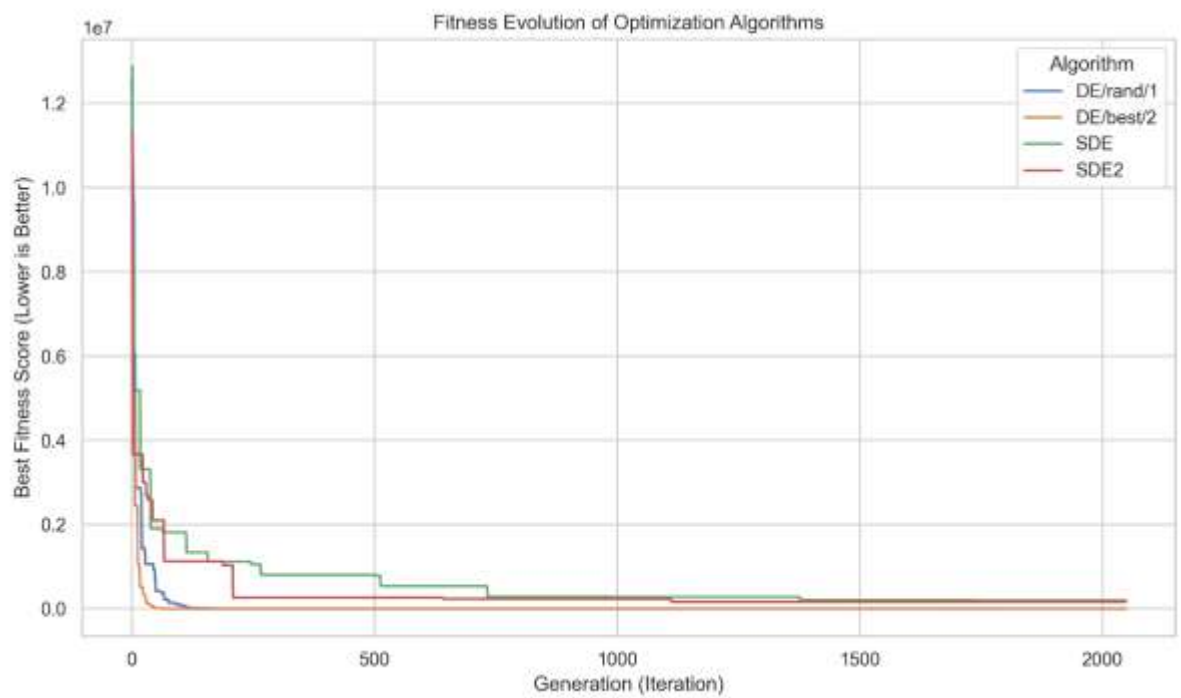
Na navedenim test funkcijama testirane su implementirane metode DE/rand/1/bin, DE/best/2/bin, SDE i modifikovana SDE2 metoda. Korišćen je uslov zaustavljanja od 2048 iteracija, a parametri su birani eksperimentalno. Za sve test funkcije su korišćeni vektori sa osam dimenzija. Za algoritme

diferencijalne evolucije praćene su vrednosti najbolje ocene i raznolikosti populacije po iteracijama. Postignute su sledeće vrednosti.

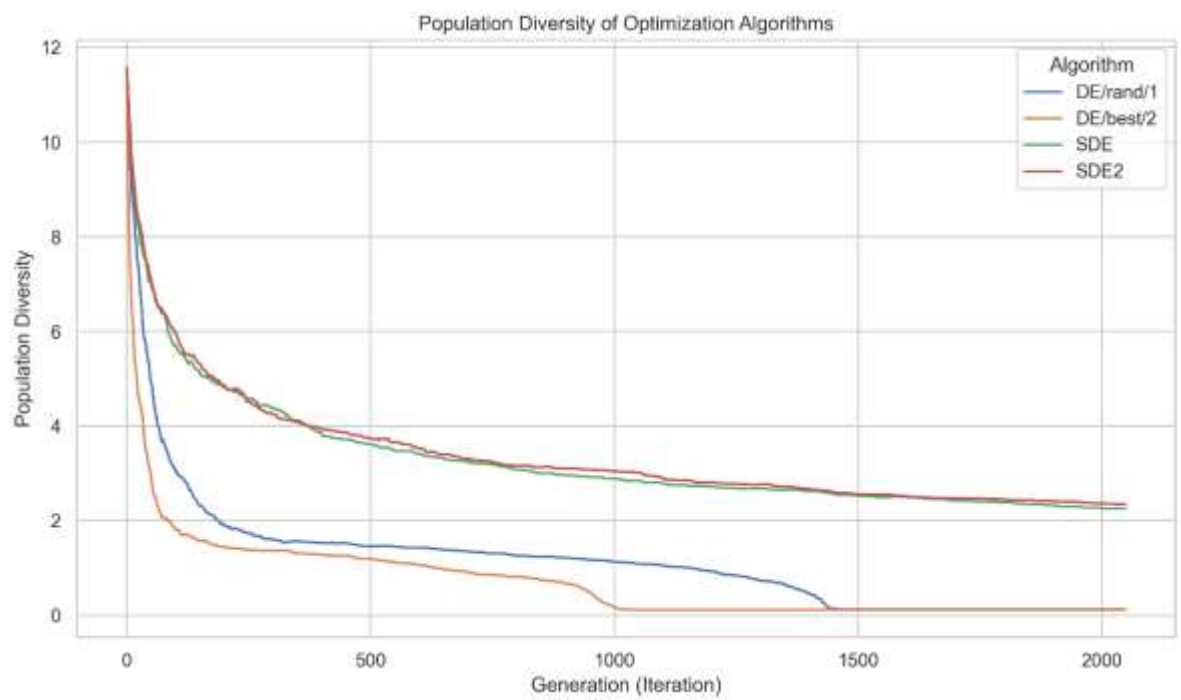
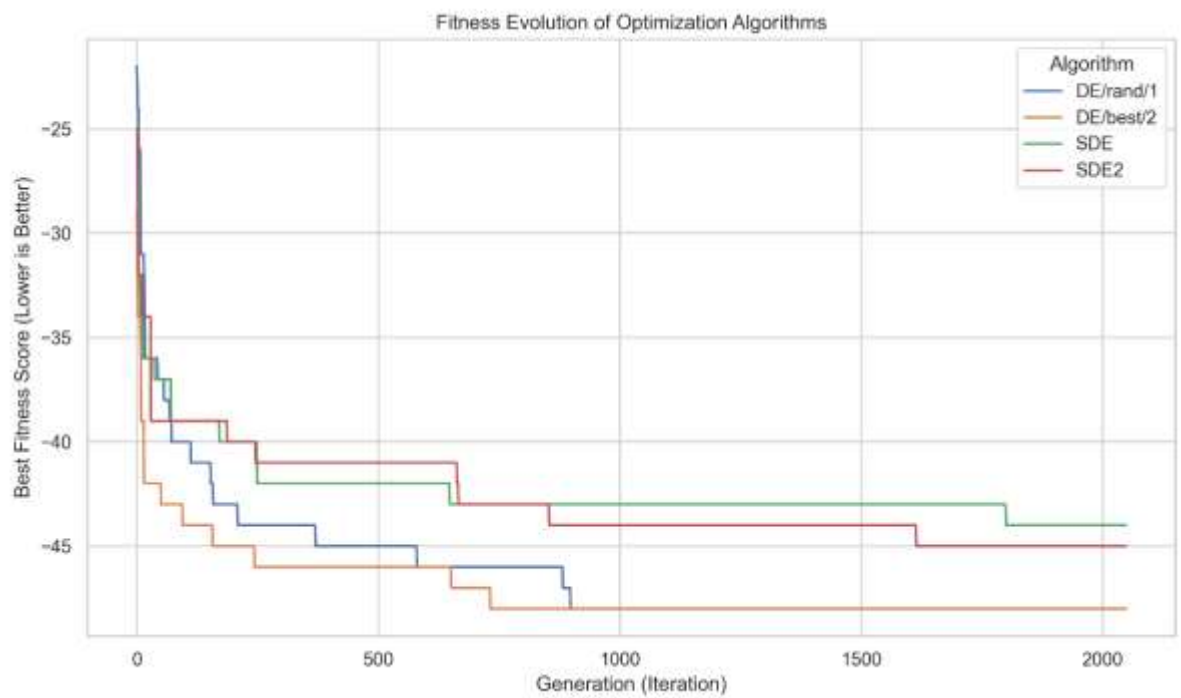
- Sphere



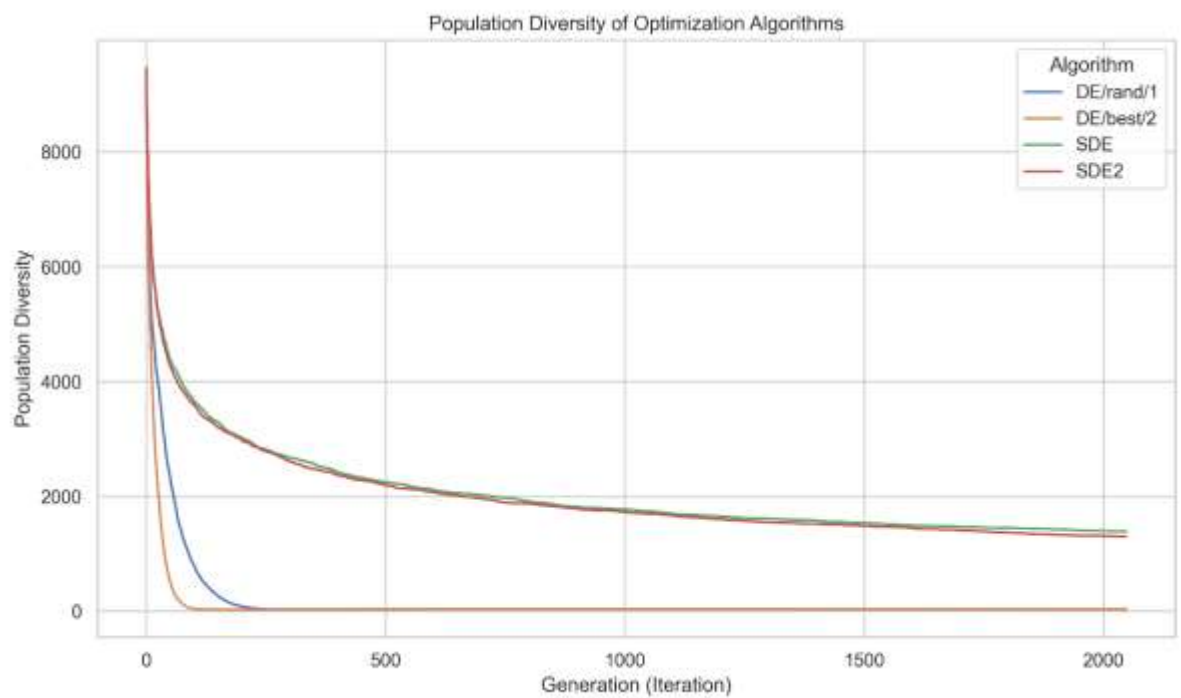
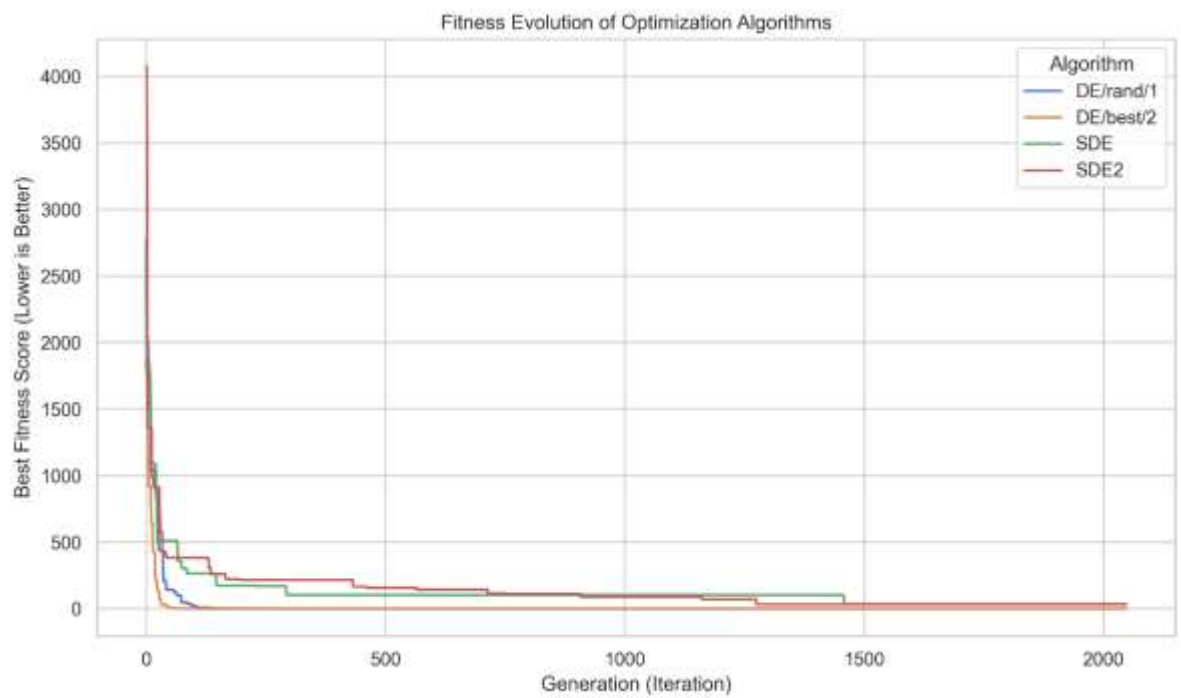
- Rosenbrock



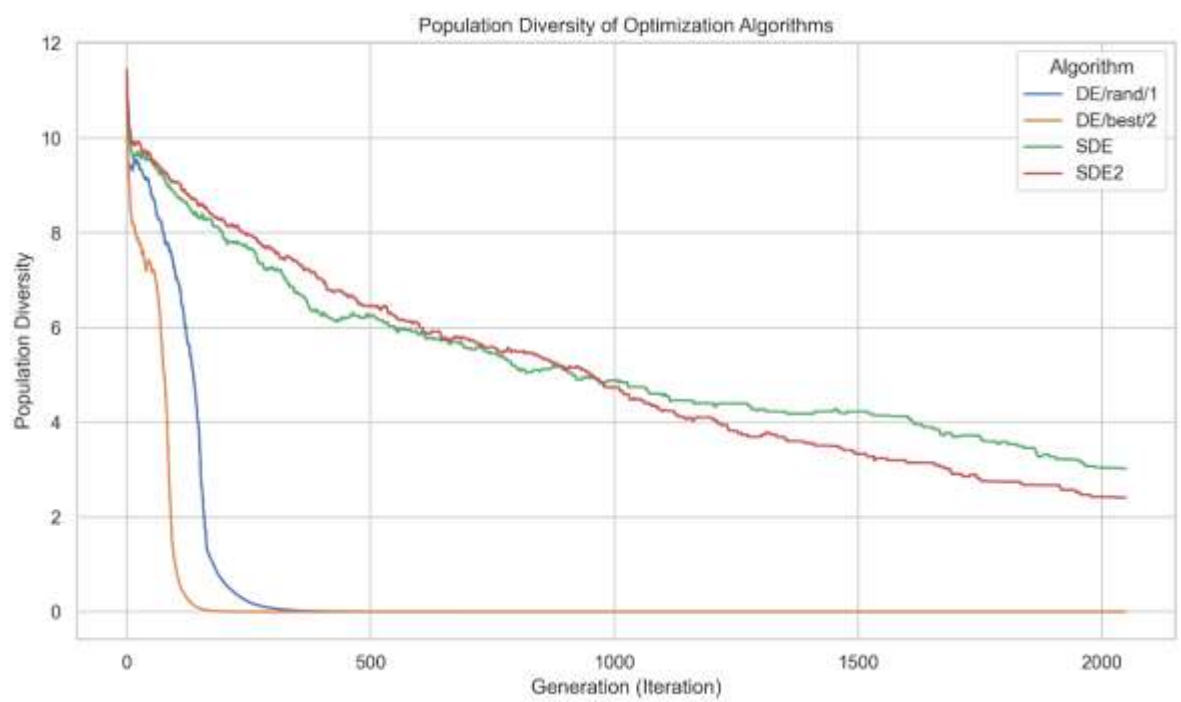
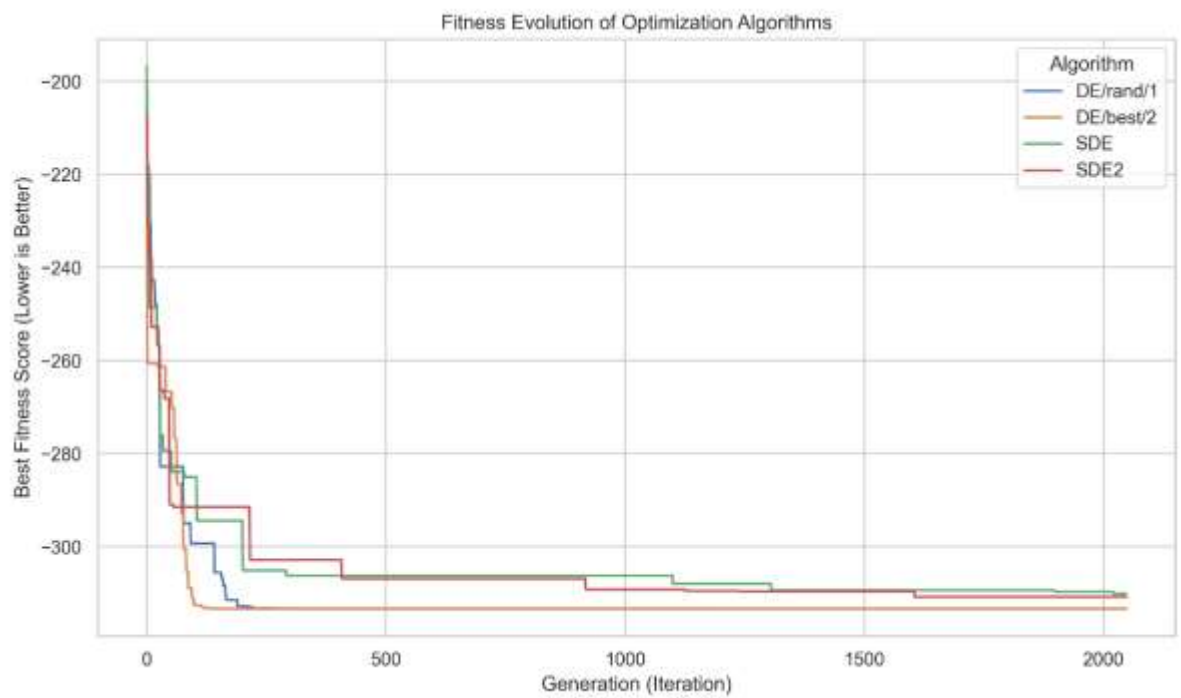
- Step



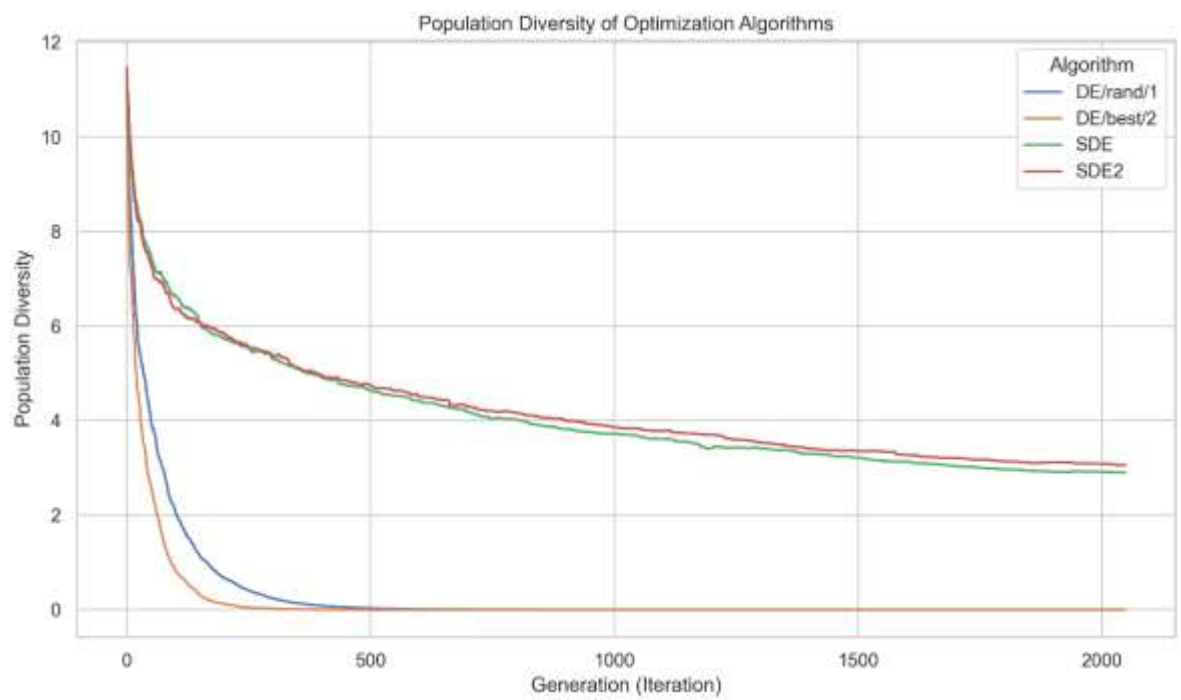
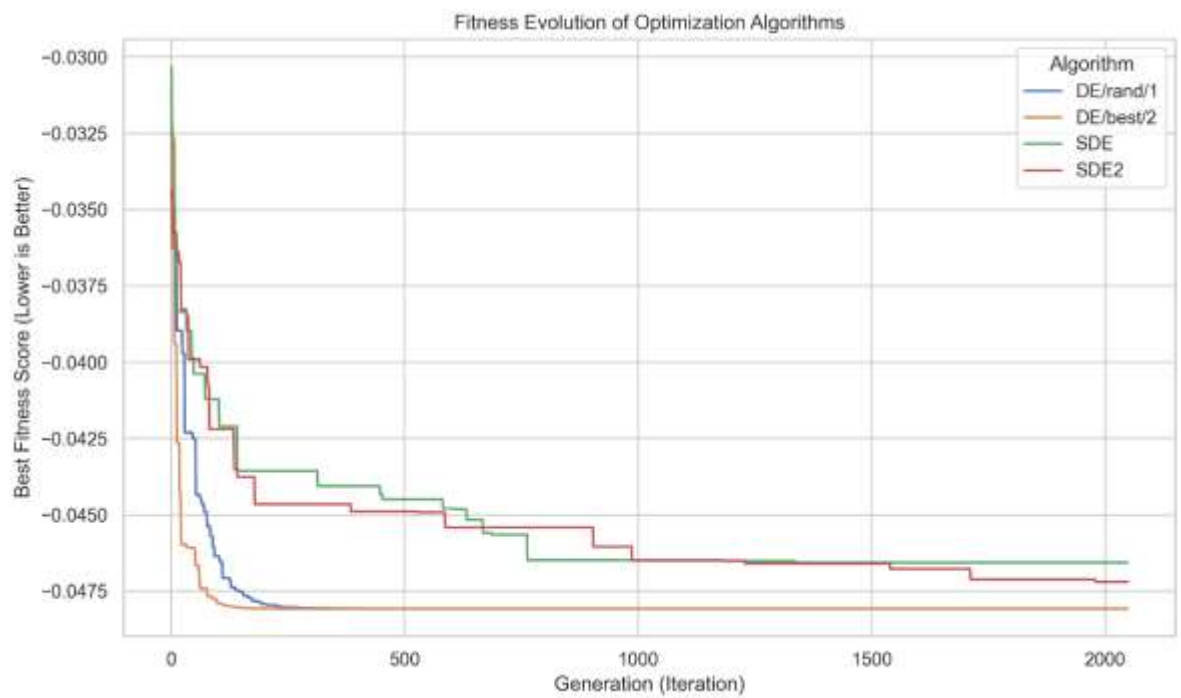
- Griewank



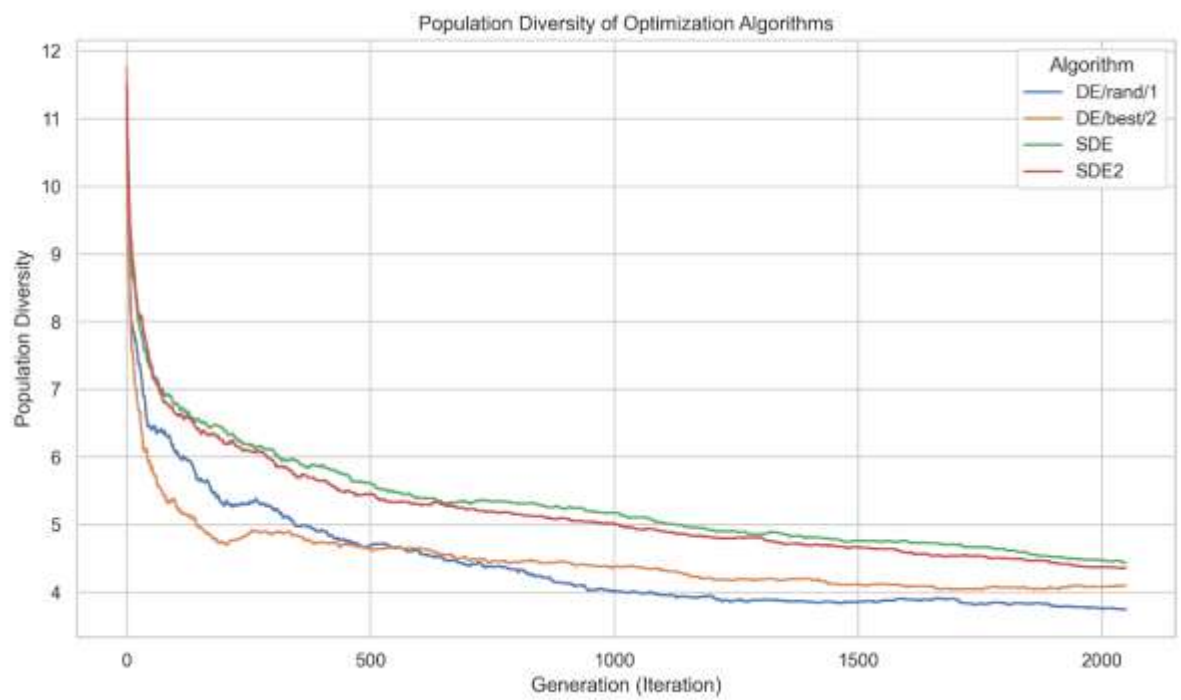
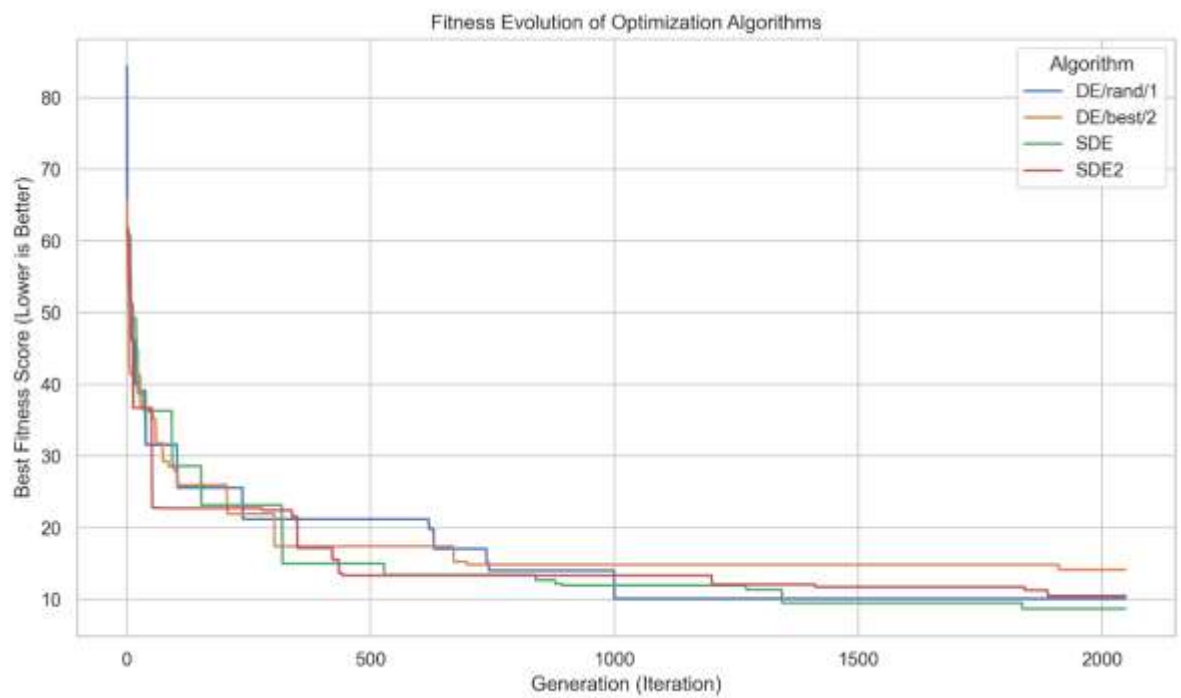
- Styblinski-Tang



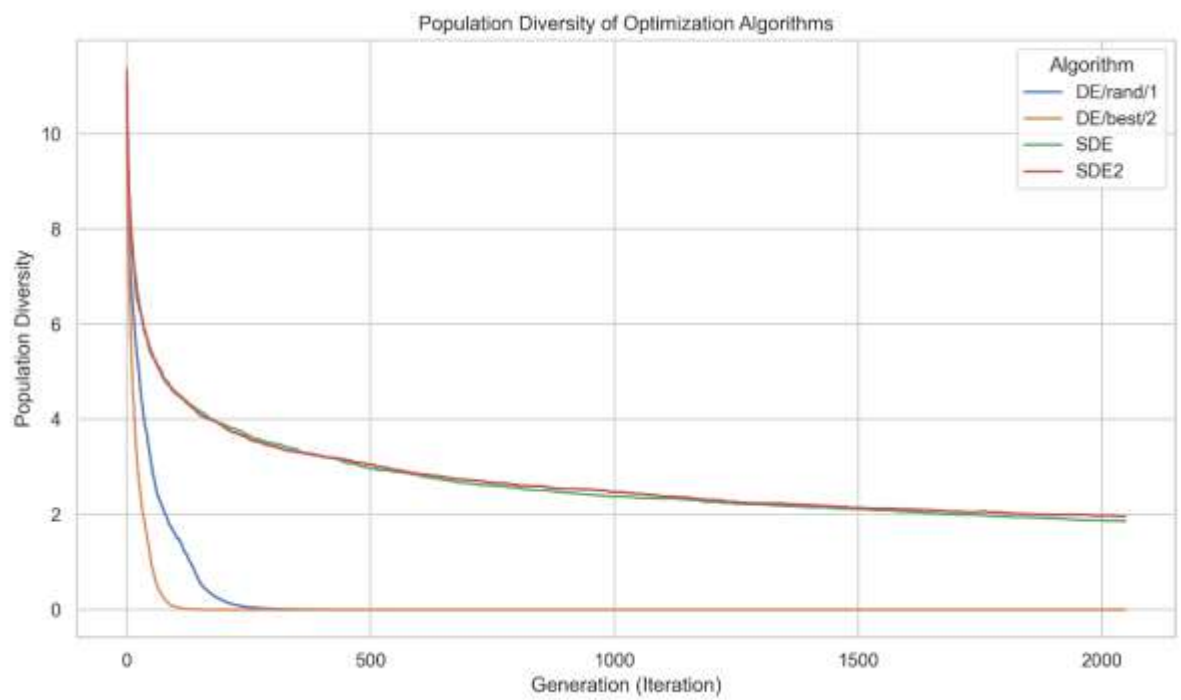
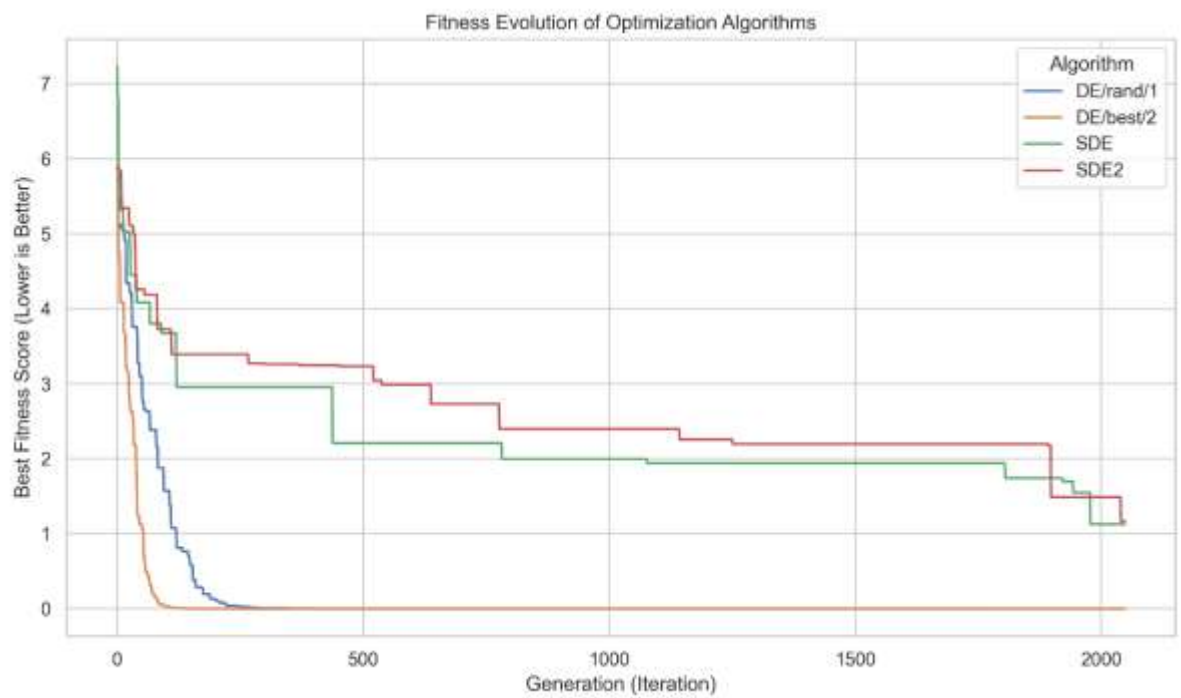
- Shekel



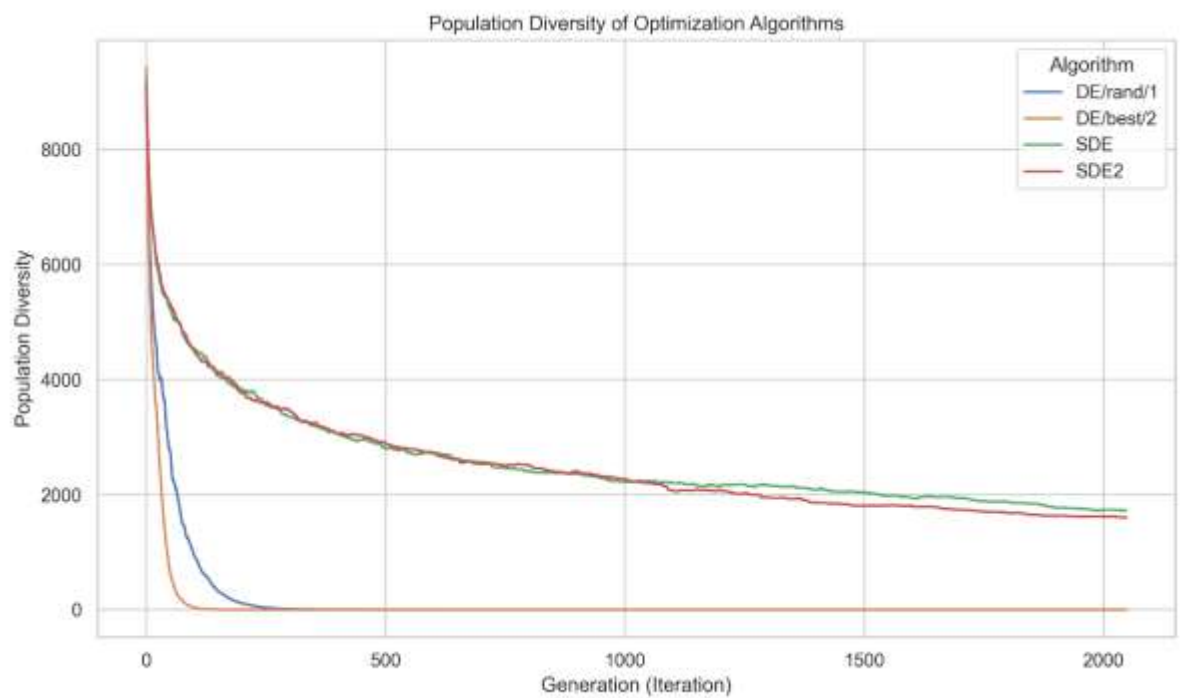
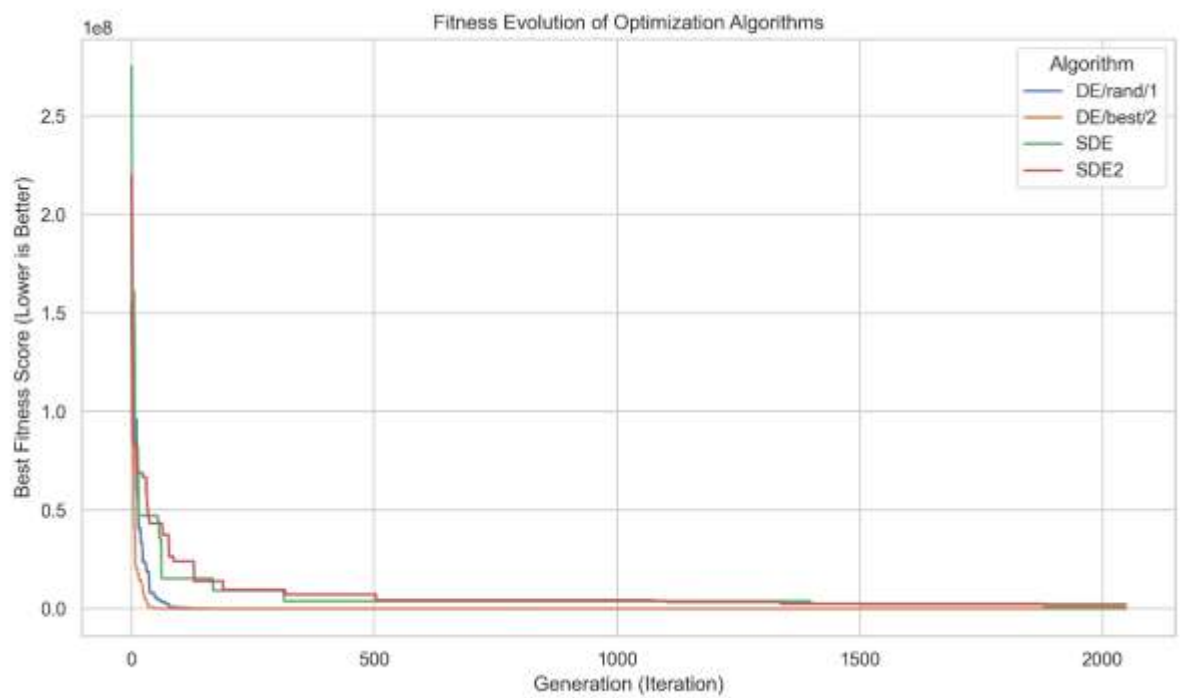
- Rastrigin



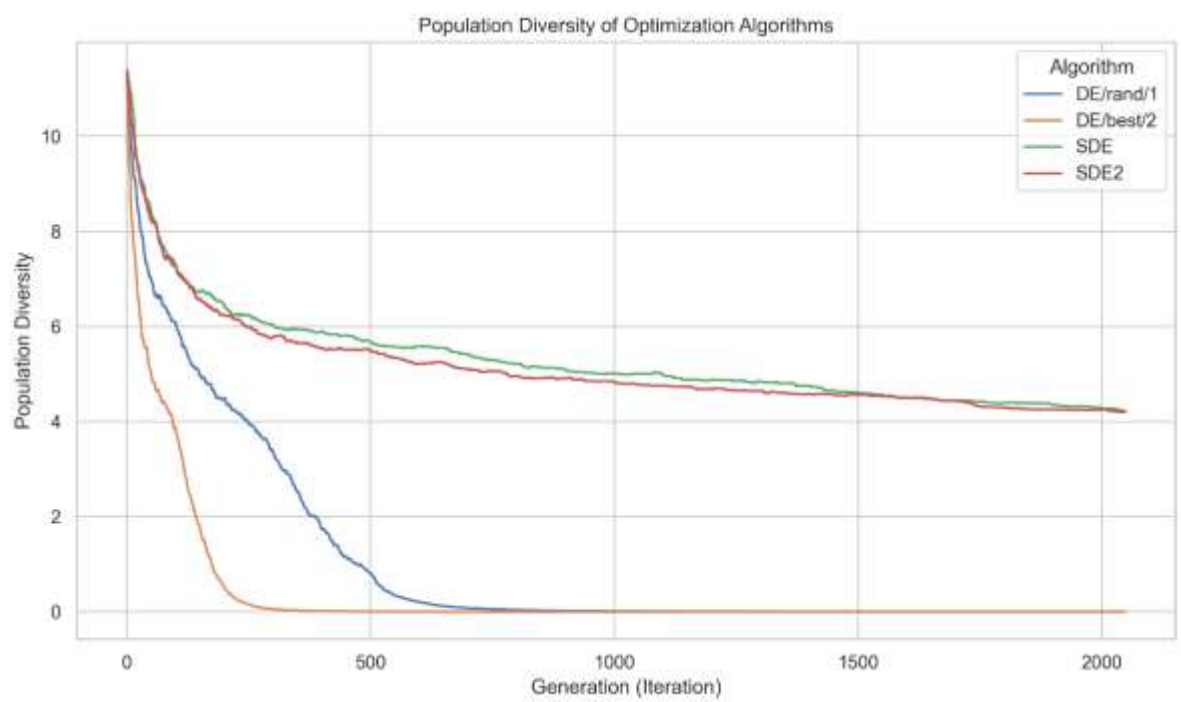
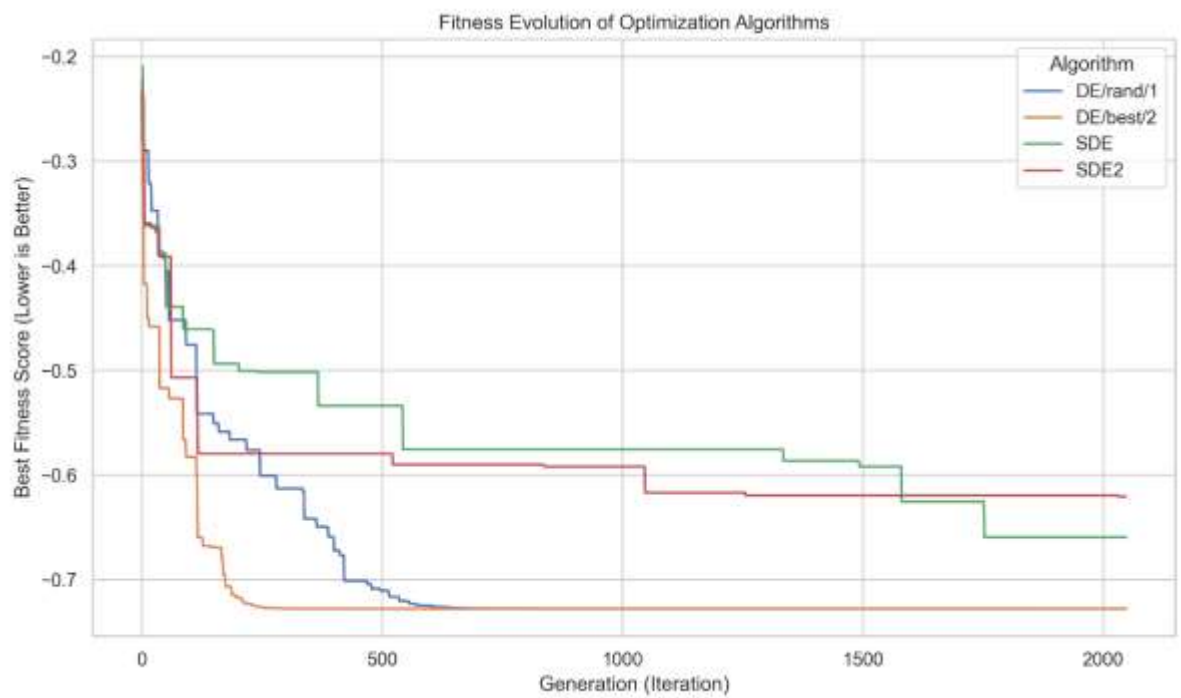
- Ackley



- Rotated Ellipsoid



- Keane's Bump



- Poređenje performansi algoritama optimizacije

Funkcija	de_rand_1	de_best_2	sde	sde2	scipy	mystic
Sphere	0.0	0.0	194134.78	163647.55	0.0	29.23
Rosenbrock	0.0	3.9859	7288576668.26	97996245297.98	9836.50	3369469.93
Step	-48	-48	-44	-45	-13.0	-13.0
Griewank	0.3632	0.2002	34.8331	32.7051	0.2041	98.2546
StyblinskiTang	-313.3293	-313.3293	-310.1191	-310.7958	-299.1926	-256.6107
Shekel	-0.0481	-0.0481	-0.0466	-0.0472	-0.0434	-0.0448
Rastrigin	10.0955	14.1168	8.7094	10.4776	84.5709	56.3484
Ackley	0.0	0.0	1.1272	1.1674	8.6664	5.1325
RotatedElipsoid	0.0	0.0	824755.20	2237954.71	16777216.0	39.5689
KeaneBump	-0.7276	-0.7276	-0.6592	-0.6206	-0.2055	-0.2251

11. Zaključak

Implementirane verzije diferencijalne evolucije uglavnom dostižu optimume. Primetno je da samoadaptivne verzije sporije konvergiraju, ali održavaju veću raznolikost populacije. Kod težih funkcija diferencijalna evolucija pokazuje bolje rezultate.

12. Reference

- [1] Feoktistov, Vitaliy. Differential Evolution: In Search of Solutions. Springer (2006).
- [2] Kochenderfer, Mykel J., and Tim A. Wheeler. Algorithms for Optimization. MIT Press (2019).
- [3] Lampinen, Jouni, and Ivan Zelinka. "Mixed Variable Non-Linear Optimization by Differential Evolution." In Proceedings of Nostradamus'99, 2nd International Prediction Conference (1999).
- [4] Storn, Rainer, and Kenneth Price. "Differential Evolution—A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces." Technical Report TR-95-012, International Computer Science Insitute (1995).
- [5] Storn, Rainer, and Kenneth Price. "Differential Evolution - A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces." Journal of Global Optimization 11, no. 4 (1997)
- [6] Price, Kenneth V., Rainer M. Storn, and Jouni A. Lampinen. Differential Evolution: A Practical Approach to Global Optimization. Natural Computing Series, Springer (2005)
- [7] Omran, Mahamed G. H., Ayed Salman, and Andries P. Engelbrecht. "Self-adaptive Differential Evolution." Evolutionary Computation (2005)
- [8] AlKhulaifi, Dania, Maryam AlQahtani, Zenab AlSadeq, Atta-ur Rahman, and Dhiaa Musleh. "An Overview of Self-Adaptive Differential Evolution Algorithms with Mutation Strategy." Mathematical Modelling of Engineering Problems 9, no. 4 (2022).
- [9] Swagatam Das, Sankha Subhra Mullick, P.N. Suganthan. "Recent advances in differential evolution – An updated survey." Swarm and Evolutionary Computation, Volume 27 (2016).